

Università degli Studi di Salerno

CORSO DI LAUREA: INGEGNERIA INFORMATICA

Software Engineering Software Design

Membri del Gruppo:

Anzivino Giuseppe Fabrizio

Borrelli Simone

Buongiorno Aldo

Calabrese Raffaele

Team Leader: Borrelli Simone

Anno Accademico 2023/2024

Indice

1. Introduzione
2. Diagramma delle Classi
 - 2.1. Definizione delle Classi
 - 2.2. Stesura del Diagramma di Classe
3. Diagrammi di Sequenza
 - 3.1. Sequence Diagram 01
 - 3.2. Sequence Diagram 02
 - 3.3. Sequence Diagram 03
 - 3.4. Sequence Diagram 04
4. Altri Diagrammi UML
 - 4.1. UI Design
 - 4.2. Diagramma di attività
 - 4.3. Diagramma di macchine a stati
5. Misura della Coesione
6. Misura dell'Accoppiamento
7. Aggiornamento della Matrice della Tracciabilità

1. Introduzione

Il gruppo ha deciso di realizzare una struttura software a Repository Condiviso, la quale prevede l'utilizzo di un solo componente che mantiene i dati persistenti. I vantaggi nell'utilizzo di tale struttura sono:

- Una facile condivisione dei dati.
- La gestione dei dati in modo centralizzato.

Gli svantaggi, invece, sono:

- La dipendenza delle componenti da questa entità
- Accesso ai dati effettuato in contemporanea da più componenti.

2. Diagramma delle Classi

2.1 Definizione delle Classi

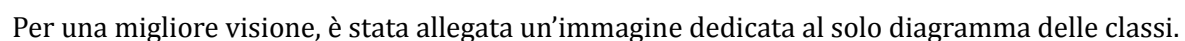
Attraverso l'utilizzo dei diagrammi di Casi d'Uso, definiti e messi per iscritto nella fase di Ingegneria dei Requisiti, sono state definite le classi, utili per la stesura del diagramma delle classi e per il corretto funzionamento dell'applicazione. Le classi definite sono:

Identificativo	Obiettivo	Funzionalità	Dipendenze
Parser	Mantenimento dei dati persistenti all'interno di una sola entità.	Traduzione e controllo della stringa passata dall'interfaccia, finalizzato a permettere l'utilizzo di tutte le altre entità.	Nessuna
Complex	Soddisfacimento dei requisiti con identificativi 1 e 3.	Creazione di un numero complesso.	Nessuna
Operation	Soddisfacimento del requisito 2 e di quelli che vanno dal 4 al 9.	Svolgimento delle operazioni aritmetiche.	Complex, Parser
Stack	Soddisfacimento del requisito 10 e di quelli che vanno dal 13 al 17.	Creazione e gestione di una pila di elementi di tipo Complex, da utilizzare per svolgere le diverse operazioni fornite dall'applicazione.	Complex, Parser
VarMap	Soddisfacimento del requisito 18 e di quelli che vanno dal 20 al 23.	Creazione e gestione di una mappa con corrispondenza tra i caratteri a-z e gli elementi di tipo Complex conservati al loro interno.	Complex, Parser
FXMLController	Soddisfacimento dei requisiti 11, 12 e 19.	Creazione di un'interfaccia utente per l'utilizzo dell'applicazione.	Parser
OverflowException	Soddisfacimento parziale del requisito 24.	Gestione di errori dovuti ad un numero troppo grande o troppo piccolo.	Da definire
ArithmeticException	Soddisfacimento parziale del requisito 24.	Gestione di errori matematici.	Da definire
FullStackException	Soddisfacimento parziale del requisito 24.	Gestione di errori dovuti ad uno stack pieno.	Da definire

EmptyStackException	Soddisfacimento parziale del requisito 24.	Gestione di errori dovuti ad uno stack vuoto.	Da definire
InputException	Soddisfacimento parziale del requisito 24.	Gestione di errori dovuti ad un input non corretto.	Da definire
VarException	Soddisfacimento parziale del requisito 24.	Gestione di errori dovuti ad una variabile già occupata o vuota a seconda dell'operazione	Da definire

2.2 Stesura del Diagramma di Classe

Dopo aver definito le diverse classi da utilizzare, il gruppo si è messo a lavoro per la realizzazione del diagramma delle classi attraverso l'utilizzo del linguaggio UML e la piattaforma Netbeans. Il diagramma risultante è il seguente:

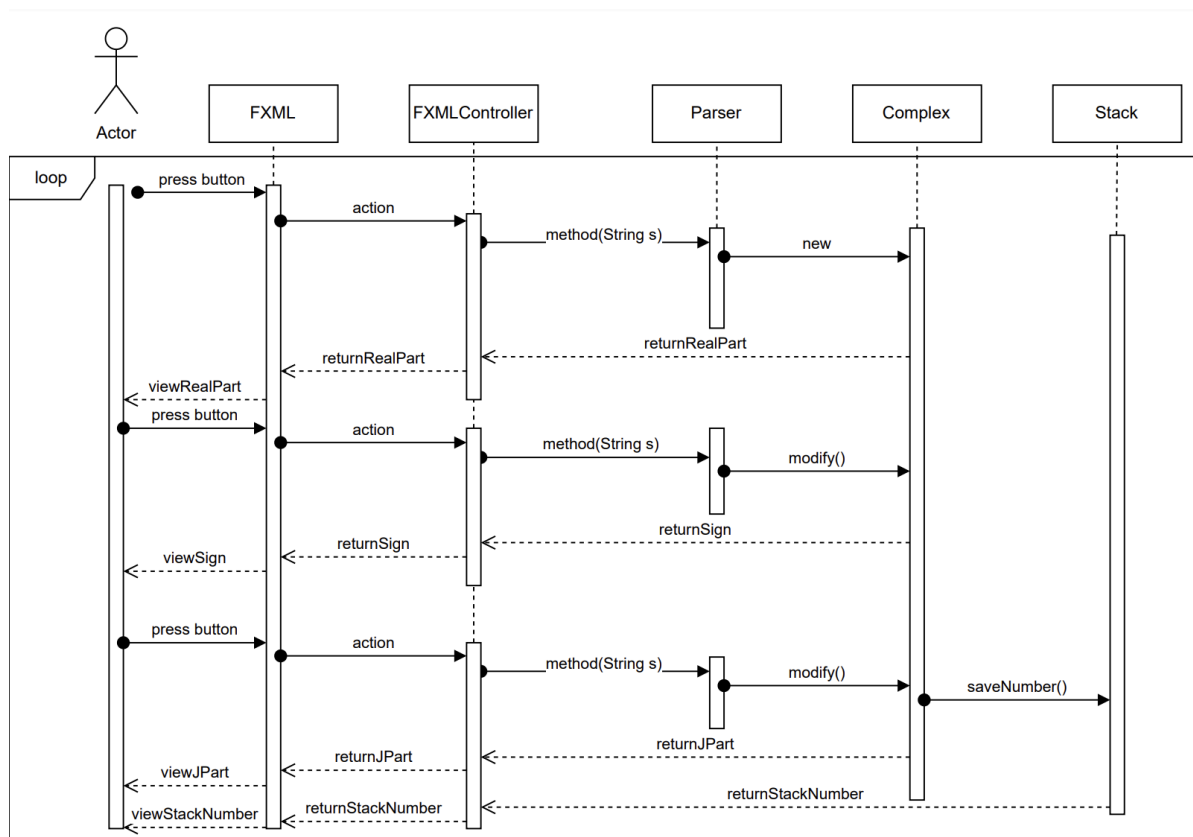


3. Diagrammi di Sequenza

In seguito alla definizione del Class Diagram e seguendo quanto descritto negli Use Cases, sono stati definiti i corrispondenti Sequence Diagrams.

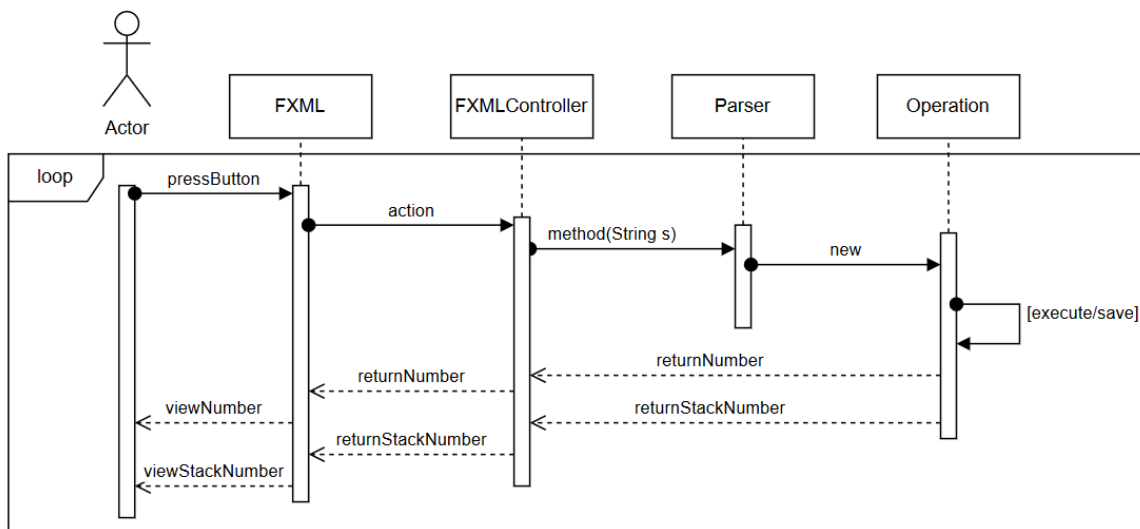
3.1 Sequence Diagram 01

Il primo sequence diagram si riferisce allo use case inerente l'inserimento degli operandi. L'utente preme il pulsante per inserire il valore desiderato. Tramite il controller il valore viene passato al traduttore (parser) che lo converte in un numero reale. La calcolatrice mostra il valore inserito sullo schermo di input. Vengono eseguite le stesse operazioni anche per l'inserimento del segno e della parte immaginaria. Il valore completo inserito dall'utente viene rappresentato sia sullo schermo di input che su quello della visualizzazione dello stack.



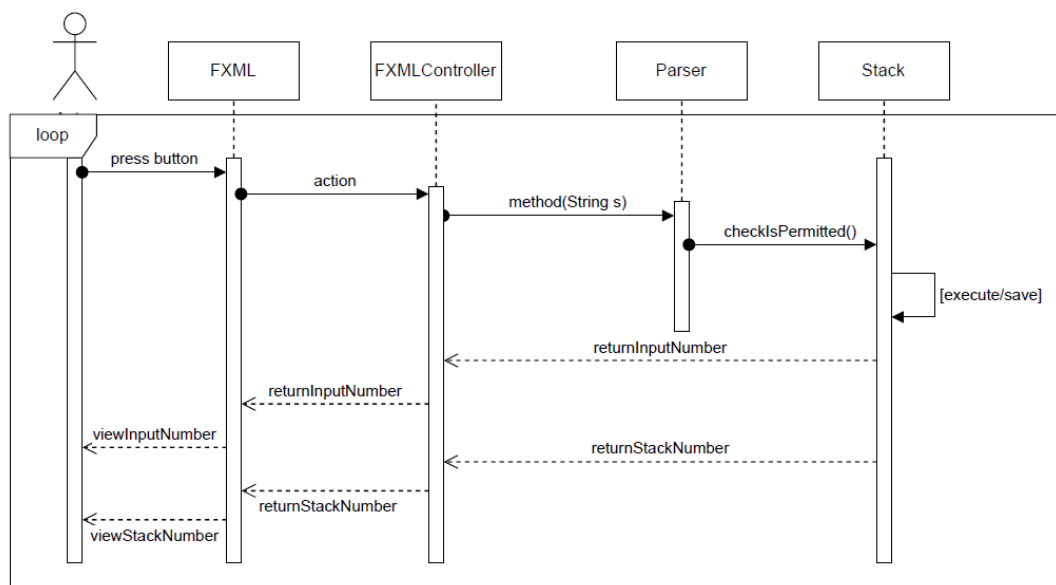
3.2 Sequence Diagram 02

Il secondo sequence diagram si riferisce allo use case inerente l'inserimento e la gestione delle operazioni. L'utente preme il pulsante per selezionare l'operazione desiderata. Tramite il controller il valore viene passato al traduttore (parser) che, tramite la classe operation, esegue il calcolo e salva il valore nello stack. Il risultato completo viene rappresentato sia sullo schermo di input che su quello di visualizzazione dello stack.



3.3 Sequence Diagram 03

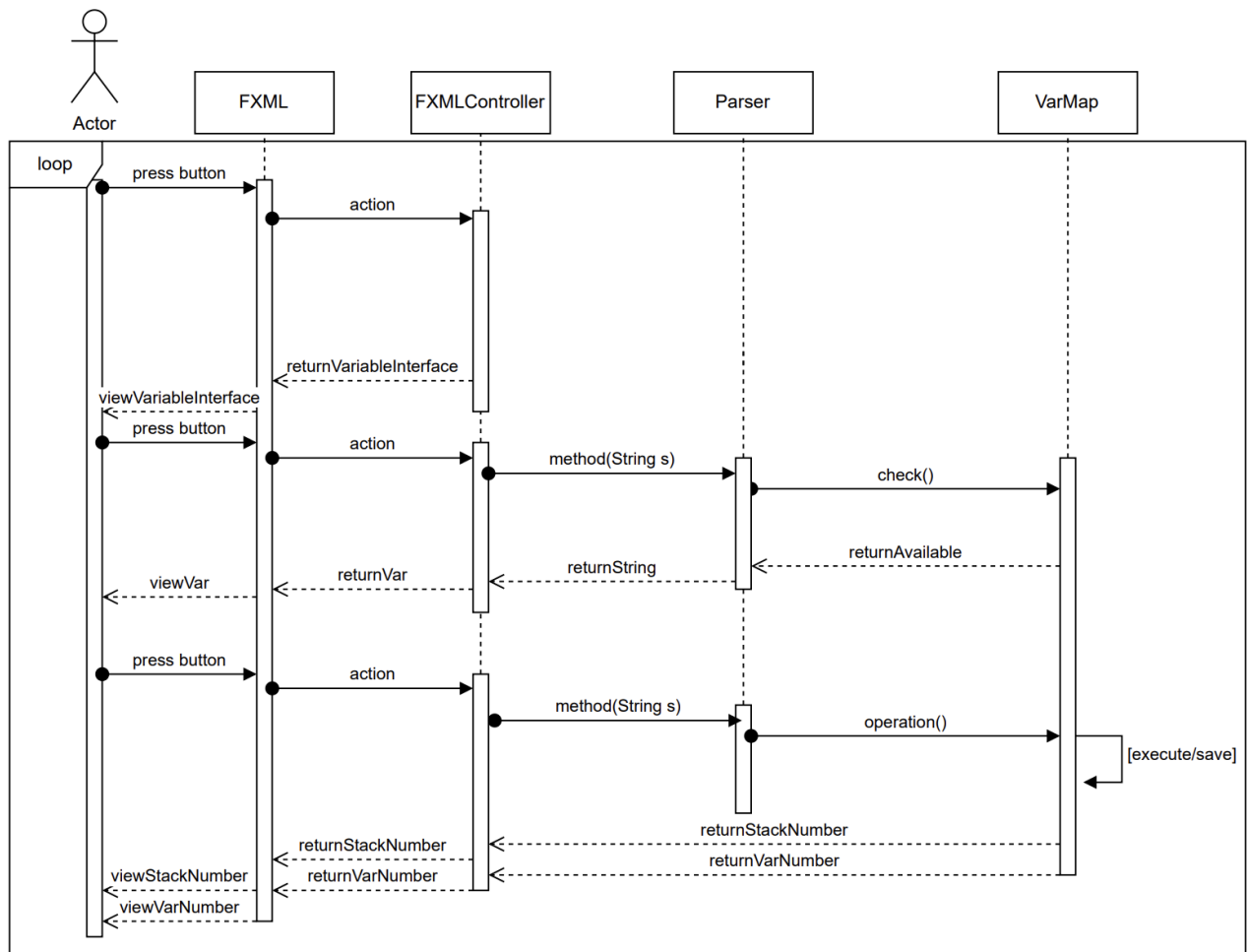
Il terzo sequence diagram si riferisce allo use case inerente la manipolazione dello stack. L'utente preme il pulsante per selezionare la funzione desiderata. Tramite il controller il valore viene passato al traduttore (parser) che, tramite lo stack, controlla se la funzione è permessa. Esegue quindi l'operazione e salva il valore all'interno dello stack. Il risultato completo viene rappresentato sia sullo schermo di input che su quello della visualizzazione dello stack.



3.4 Sequence Diagram 04

Il quarto sequence diagram illustra quanto descritto nello use case 04, relativo alla gestione delle variabili.

L'utente preme il pulsante per passare all'interfaccia delle variabili, cosicché venga mostrata l'interfaccia delle variabili. L'utente preme il pulsante della variabile scelta e tramite il controller viene passata al traduttore (parser), che controlla se tale variabile è accessibile. Preme quindi il pulsante dell'operazione desiderata e, tramite prima il controller e poi il traduttore, viene eseguita l'operazione e il risultato completo viene mostrato sia sullo schermo delle variabili che su quello della visualizzazione dello stack.

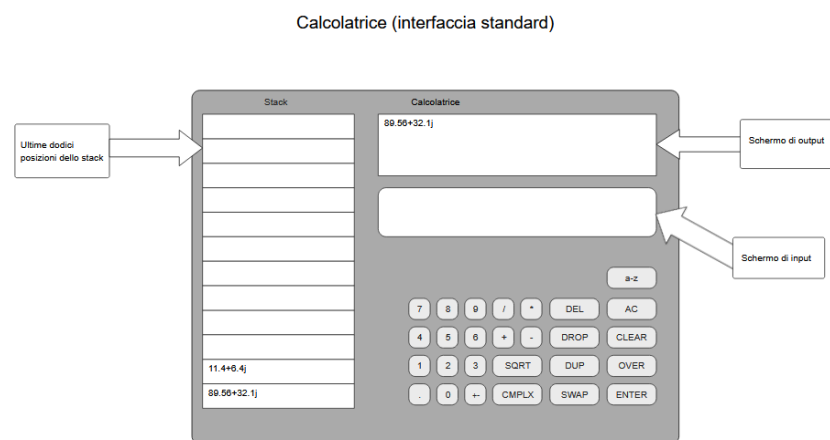


4. Altri Diagrammi UML

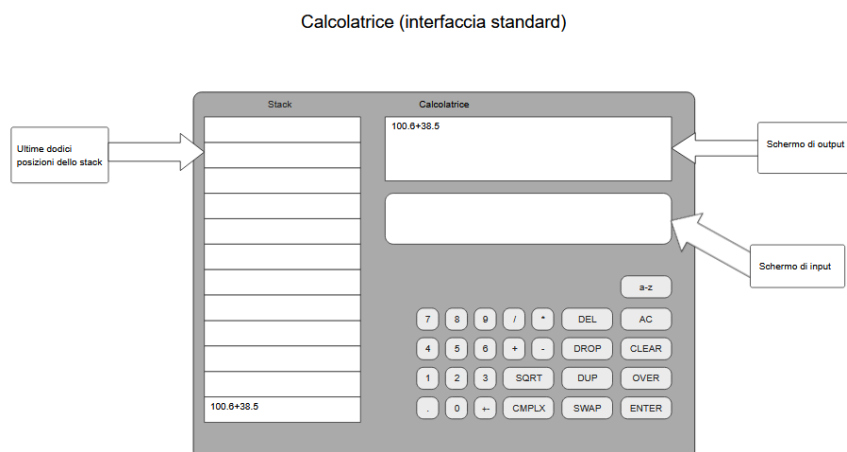
In seguito alla stesura dei due principali diagrammi della fase di Design, sono stati creati altri diagrammi, utili a dare una visione ancora più chiara della struttura generale della progettazione.

4.1 UI Design

Partendo dalla prototipazione svolta nella fase di Ingegneria dei Requisiti, bisogna definire come cambia l'interfaccia a seconda del flusso logico. Nel momento in cui viene inserito un numero complesso, l'interfaccia cambia nel seguente modo:

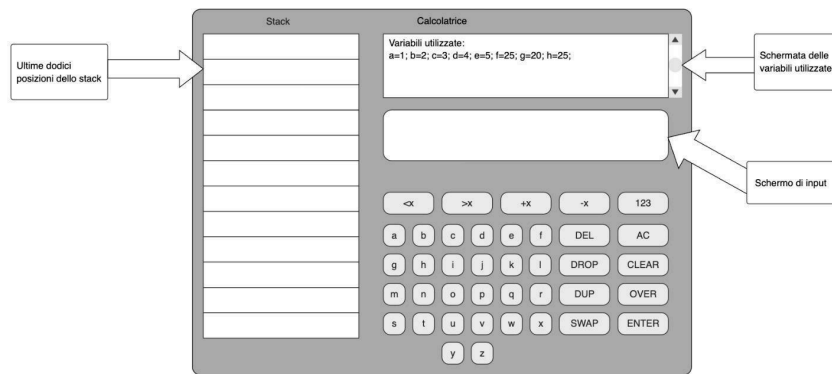


Nel momento in cui si voglia svolgere un'operazione con i numeri complessi presenti all'interno dello stack, l'interfaccia cambia nel seguente modo (in questo esempio viene eseguita la somma con i valori visibili nell'esempio precedente):



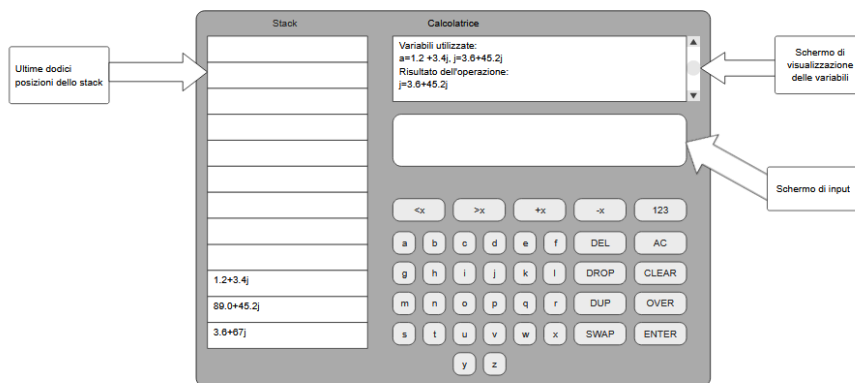
Nel momento in cui viene digitato il tasto a-z si passa all'interfaccia apposita per l'utilizzo delle variabili (come abbiamo già visto nella fase di Ingegneria dei Requisiti):

Calcolatrice (interfaccia variabili)



Nel momento in cui viene digitato un'operazione con le variabili, l'interfaccia cambia nel seguente modo (esempio di stackToVar):

Calcolatrice (interfaccia variabili)



Nel momento in cui si digita il tasto drop, l'interfaccia cambia nel seguente modo (nell'esempio consideriamo uno stack riempito come quello dell'esempio precedente):

Calcolatrice (interfaccia variabili)



Nel momento in cui si digita il tasto swap, l'interfaccia cambia nel seguente modo(nell'esempio consideriamo uno stack riempito come quello dell'esempio precedente):



Nel momento in cui si digita il tasto dup, l'interfaccia cambia nel seguente modo(nell'esempio consideriamo uno stack riempito come quello dell'esempio precedente):

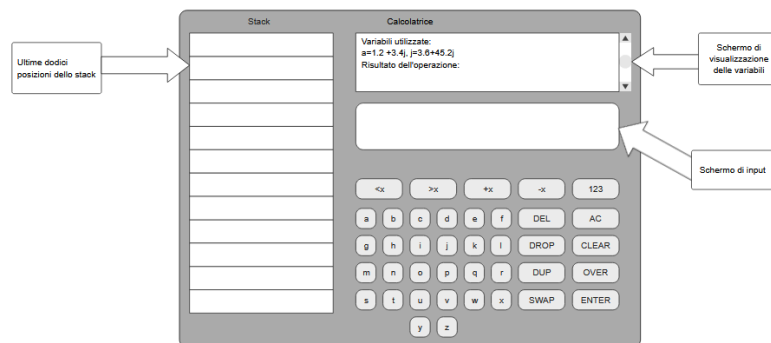


Nel momento in cui si digita il tasto over, l'interfaccia cambia nel seguente modo(nell'esempio consideriamo uno stack riempito come quello dell'esempio precedente in seguito al tasto drop digitato):



Nel momento in cui si digita clear, l'interfaccia cambia nel seguente modo(nell'esempio consideriamo uno stack riempito come quello dell'esempio precedente):

Calcolatrice (interfaccia variabili)

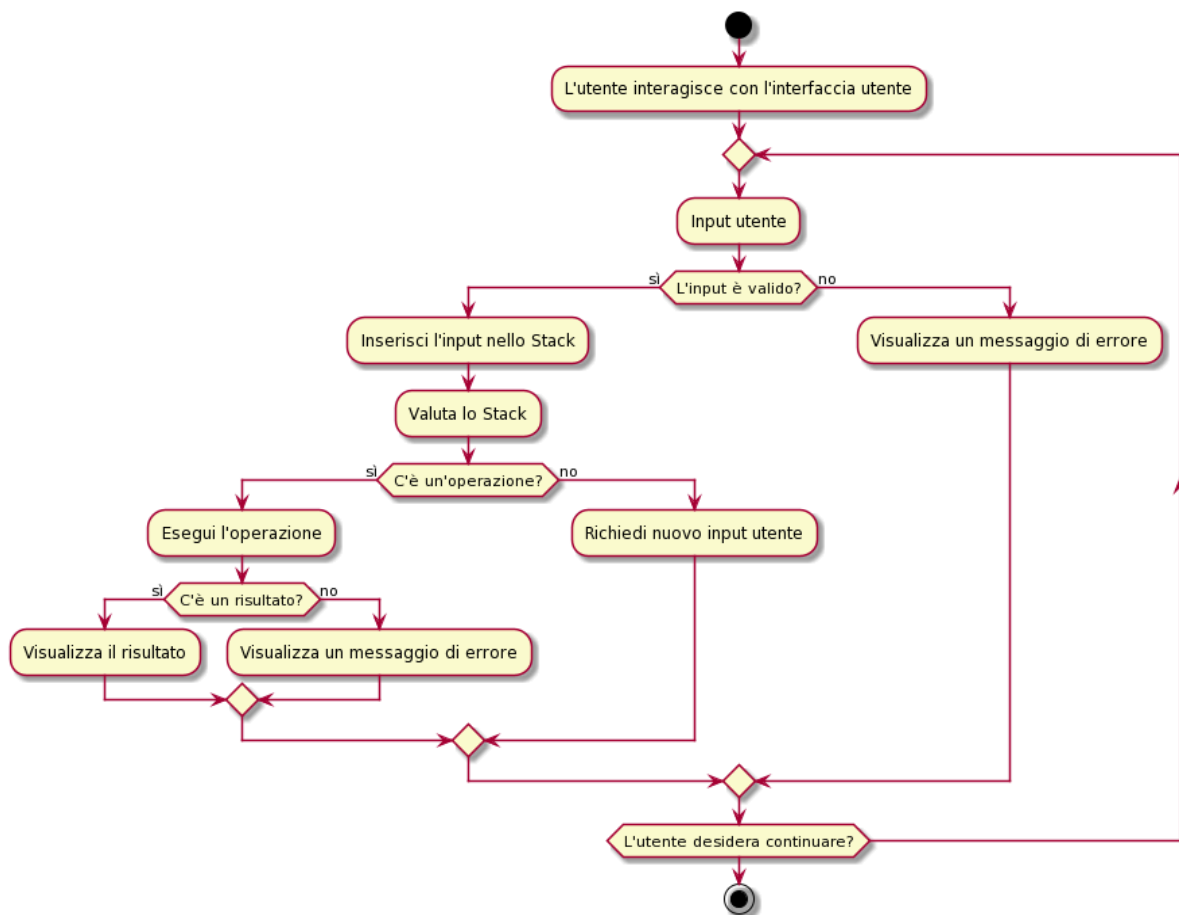


4.2 Diagramma di Attività

Per la creazione di un diagramma di attività, utile per la descrizione del flusso di interazione del nostro progetto, il gruppo ha seguito questi passaggi generali:

- Identificare le Attività Principali:
 - Inserimento di Input: L'utente inserisce dati attraverso la tastiera.
 - Esecuzione di Calcoli: Il sistema interpreta gli input e esegue i calcoli.
 - Visualizzazione del Risultato: Il risultato viene mostrato all'utente.
- Identificare le Decisioni:
 - Decisione su Operazione: Determinare l'operazione da eseguire (addizione, sottrazione, ecc.).
- Aggiungere Blocchi di Azioni:
 - Interpretazione Input: Converti gli input dell'utente in oggetti comprensibili.
 - Gestione Stack: Effettua operazioni di push, pop e manipolazioni dello stack.
 - Operazioni Matematiche: Effettua le operazioni matematiche richieste.
- Collegare i Blocchi:
 - Usare frecce direzionali per collegare i blocchi, indicando il flusso delle attività.
 - Utilizzare diamanti (decisioni) per le condizioni e connettere le frecce di conseguenza.

In seguito a questi passaggi è stato realizzato il seguente diagramma:

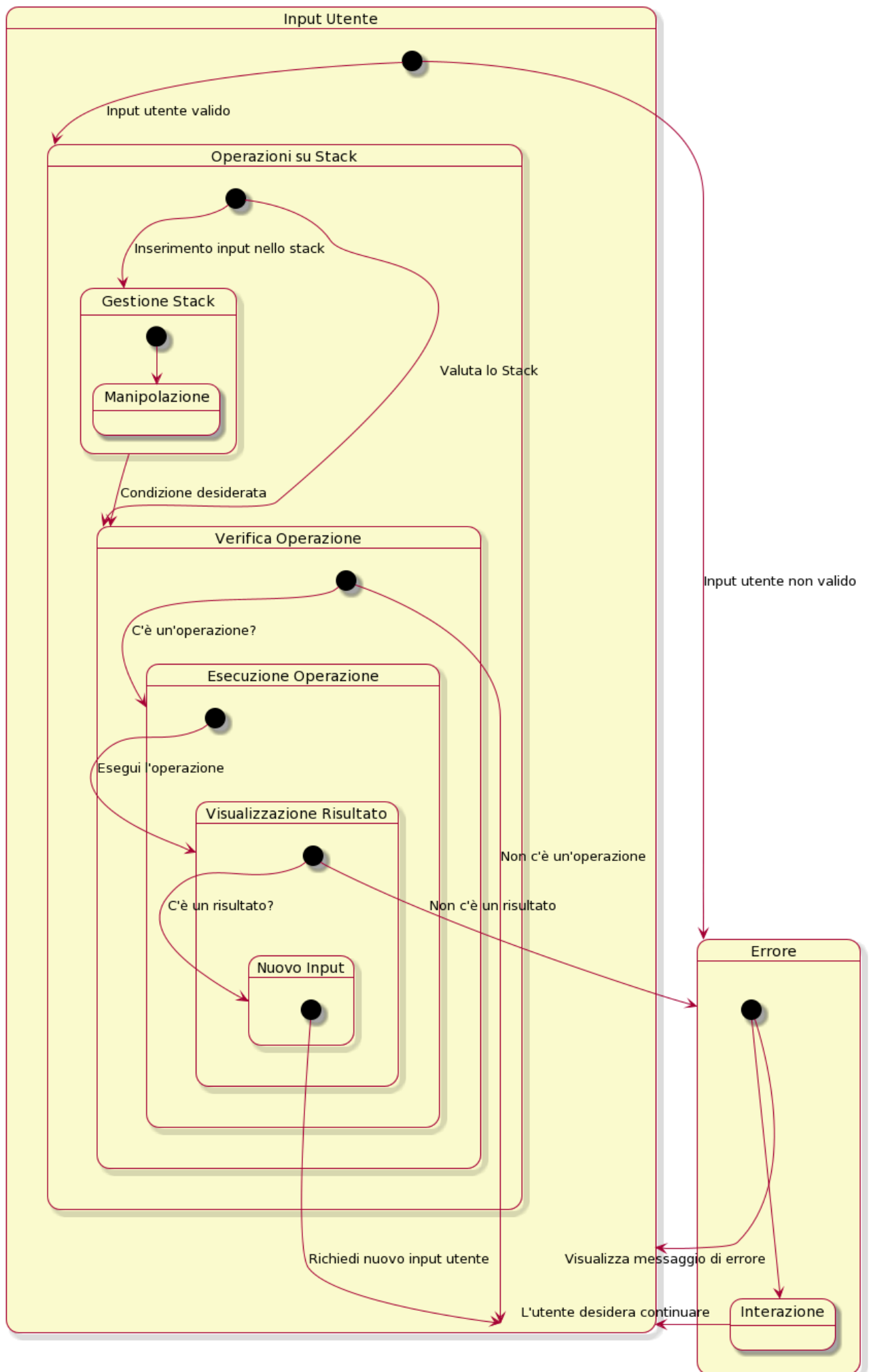


4.3 Diagramma di macchine a stati

Inoltre è stato realizzato anche un diagramma di macchine a stati che rappresenta il comportamento della nostra applicazione. Ecco una descrizione degli Stati, nei quali la macchina inizia con lo stato "Attesa Input Utente" (S0), che è lo stato principale in cui l'applicazione attende l'input da parte dell'utente.

- **Input Utente (S1)**, nel quale dall'attesa dell'input utente (Interazione), si passa allo stato "Input Utente" (S1). Se l'input è valido, la macchina passa a "Operazioni su Stack" (S2), altrimenti ritorna all'attesa dell'input (S0) in caso di input non valido.
- **Operazioni su Stack (S2)**, nel quale si eseguono due azioni parallele, cioè l'inserimento dell'input nello stack (S3) e la valutazione dello stack (S4).
- **Gestione Stack (S3)**, nel quale vengono eseguite azioni relative alla manipolazione dello stack.
- **Verifica Operazione (S4)**, nel quale si controlla se c'è un'operazione da eseguire: se non c'è un'operazione, si torna allo stato "Input Utente" (S1), altrimenti, si passa allo stato "Esecuzione Operazione" (S5).
- **Esecuzione Operazione (S5)**, nel quale viene eseguita l'operazione identificata.
- **Visualizzazione Risultato (S6)**, nel quale si controlla se c'è un risultato da mostrare: se c'è un risultato, lo si mostra e si passa allo stato "Nuovo Input" (S7), altrimenti, si ritorna all'attesa dell'input utente (S0).
- **Nuovo Input (S7)**, nel quale si chiede un nuovo input all'utente. Successivamente, si torna allo stato "Input Utente" (S1).
- **Errore (S8)**, nel quale si visualizza un messaggio di errore e si ritorna all'attesa dell'input utente (S1), quindi "Errore" è visto come una versione alternativa di S0.

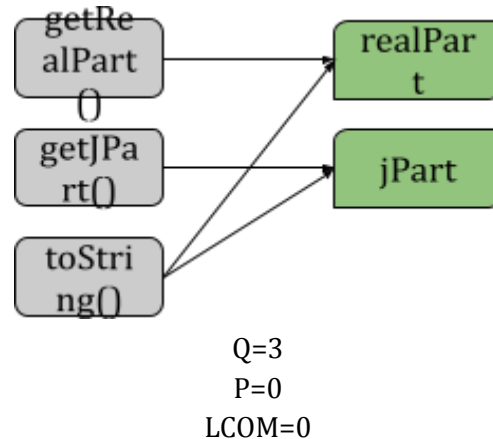
Per quanto riguarda il Flusso Generale, l'applicazione cicla tra l'attesa dell'input, l'inserimento nello stack, la valutazione dello stack, l'esecuzione dell'operazione e la visualizzazione del risultato. Inoltre, l'utente può decidere se continuare fornendo nuovo input o interrompere l'esecuzione.



5. Misura della Coesione

La coesione indica quanto le funzioni di ogni componente sono legate fra loro. Tale coesione viene misurata all'interno delle classi usando il metodo LCOM, il quale definisce che la coesione all'interno di una classe o in un modulo di codice equivale a $\max(P-Q, 0)$. In tale equazione Q indica il numero di coppie di metodi all'interno della classe che accedono ad uno o più attributi appartenenti alla stessa classe, mentre P indica il numero di coppie di metodi all'interno della classe che non accedono agli attributi condividendoli. Prendendo in considerazione il diagramma delle classi, è stato fatto il calcolo della coesione per ognuna delle classi definite:

- Classe complex presenta la seguente situazione:



- Classe Operation presenta una situazione nella quale tutti i metodi vanno a fare uso dello stack, di conseguenza non esistono coppie di metodi che non fanno uso di attributi che vengono condivisi. Di conseguenza, LCOM è pari a 0.
- Classe Stack presenta la stessa medesima situazione della classe Operation
- Classe VarMap presenta la stessa situazione corrispondente alla classe Operation e la classe Stack secondo la quale non ci sono coppie di metodi che non condividano gli attributi.
- Classe Parser presenta una situazione nella quale vi è solo il metodo parse() che accede agli attributi della classe e di conseguenza non è possibile parlare di coppie e LCOM sarà un valore nullo.
- Classe FXMLController presenta una situazione nella quale tutti i metodi accedono a tutti gli attributi portando così a non avere coppie che fanno uso di attributi indipendentemente dagli altri. Di conseguenza anche in questo caso LCOM è pari a valore nullo.

In conclusione la coesione del codice da dover implementare è praticamente nulla. Ciò è dovuto alla decisione presa dal gruppo nell'utilizzare un Repository Condiviso come struttura del software, il quale ha come svantaggio l'utilizzo dei dati persistenti da più componenti.

6. Misura dell'Accoppiamento

L'accoppiamento indica quanto i vari componenti dipendono da altri componenti. Tale indicatore, considerando due entità x e y , viene calcolato come $C(x, y) = I + N / (N + 1)$. In tale calcolo I indica il livello di accoppiamento tra due entità e N indica il numero di relazioni tra le due entità. I diversi livelli di accoppiamento sono:

1. Data Coupling, vengono passati semplicemente i dati tra due entità.
2. Stamp Coupling, vengono passati dati in più di quelli che servono.
3. Control Coupling, vengono passate informazioni di controllo influenzandone la logica.
4. Common Coupling, vengono condivisi gli stessi dati da due entità facendo riferimento alla stessa variabile globale.
5. Content Coupling, due entità accedono ai dati interni reciprocamente e la modifica di un'entità implica una revisione dell'altra.

Prendendo in considerazione il diagramma delle classi, il gruppo ha definito il calcolo dell'accoppiamento per ogni coppia di classe:

- Tra Parser e Complex ci sono due relazioni e il livello di accoppiamento tra i due è il primo. Di conseguenza $C(\text{Parser}, \text{Complex}) = 1 + \frac{2}{3}$.
- Tra Parser e Stack c'è una sola relazione e il livello di accoppiamento è il quarto. Di conseguenza $C(\text{Parser}, \text{Stack}) = 4 + \frac{1}{2}$. Stesso discorso per l'accoppiamento tra:
 - Parser e VarMap
 - Parser e Operation
 - Parser e FXMLElementController
- Tra VarMap e Complex ci sono tre relazioni e il livello di accoppiamento è il primo. Di conseguenza $C(\text{VarMap}, \text{Complex}) = 1 + \frac{3}{4}$. Discorso analogo per l'accoppiamento tra FXMLElementController e Complex.
- Tra Operation e Complex vi è una sola relazione e il livello di accoppiamento è il primo. Di conseguenza $C(\text{Operation}, \text{Complex}) = 1 + \frac{1}{2}$.
- Tra Operation e VarMap ci sono due relazioni e il livello di accoppiamento è il quarto. Di conseguenza $C(\text{Operation}, \text{VarMap}) = 4 + \frac{2}{3}$. Discorso analogo per l'accoppiamento tra:
 - Operation e Stack
 - Operation e FXMLElementController
 - VarMap e Stack
 - VarMap e FXMLElementController
 - Stack e FXMLElementController

In conclusione, l'accoppiamento è il più delle volte troppo alto a causa dello stack che viene utilizzato come variabile globale dalla maggior parte delle classi. Tale indicatore è troppo alto a causa della scelta della struttura software fatta dal gruppo, in quanto il Repository Condiviso ha come svantaggio la dipendenza delle diverse componenti da un'entità utilizzata per la persistenza dei dati.

7. Aggiornamento della Matrice della Tracciabilità

Legenda:



: Requisito non ancora soddisfatto nella fase corrispondente.



: Requisito soddisfatto nella fase corrispondente.

ID : Identificativo del paragrafo dove viene soddisfatto il Requisito.

Requisito	Design	Implementazione	Testing	Requisiti Correlati
1	 ID: 2. e 3.1			...
2	 ID: 2. e 3.2			1
3	 ID: 2.			1
4	 ID: 2.			1, 2, 3
5	 ID: 2.			1, 2, 3
6	 ID: 2.			1, 2, 3
7	 ID: 2.			1, 2, 3
8	 ID: 2.			1, 2, 3
9	 ID: 2.			1, 2, 3
10	 ID: 2. e 3.3			1, 2, 3
11	 ID: 4.1			1-10
12	 ID: 4.1			1, 2, 11

13	 ID: 2.			10, 11
14	 ID: 2.			10, 11
15	 ID: 2.			10, 11
16	 ID: 2.			10, 11
17	 ID: 2.			10, 11
18	 ID: 2. e 3.4			1, 2, 10, 11
19	 ID: 4.1 e 3.			12, 18
20	 ID: 2.			1, 2, 10, 11, 18, 19
21	 ID: 2.			1, 2, 10, 11, 18, 19
22	 ID: 2.			1, 2, 10, 11, 18, 19
23	 ID: 2.			1, 2, 10, 11, 18, 19
24	 ID: 2.			1, 2, 10, 18, 19
25	 ID: 4.1 e 4.2			1, 2, 10, 11, 12, 18, 19
26	 ID: 2. e 3.			...
27	 ID: 2.			Tutti